

This course has already ended.

« Chapter 18.2: Types in classes, Mixin and Self Type ...

Round 19 » (https://plus.cs.aalto.fi/studio_2/k2022/w...

CS-C2120 (https://plus.cs.aalto.fi/studio_2/k2022/) / Round 18 (https://plus.cs.aalto.fi/studio_2/k2022/w18/)
/ chapter 18.3: Modifying expressions

chapter 18.3: Modifying expressions

About this page:

Topics We introduce trees and how expressions can be presented in tree form.

What Will I Do? First reading and then programming.

Indicative difficulty level: It's easy when you get the idea.

Rough Estimate of Workload:? 2-3 hours.

Point Value: 200 (20+20+60+100)

Related

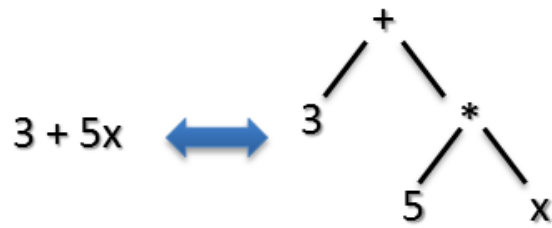
Projects:

Expressions 

(https://gitmanager.cs.aalto.fi/static/studio2_k2022dev2-horrible-gitmanager-interface/projects/given/Expressions/).

Expressions and trees

AST, or abstract syntax tree  (http://en.wikipedia.org/wiki/Abstract_syntax_tree) is a tree-like presentation of program code. It has a lot of use in verifying the correctness, optimizing, and compiling code of programs.



Some terminology

Tree is a hierarchical data structure consisting of **nodes**. The simplest tree has only one node, **root node**. However, the root node may have **other nodes** as **children**, which are trees. They may still have children, etc. A node with no children is called a **leaf node**.

In contrast to life outside computers, the trees used by programmers are growing downwards (you can compare the idea with family trees).

Exercises 3-6

In this exercise, we construct mathematical expressions from nodes, calculate the values of the expressions, simplify them, and finally count the expression derivative.

Nodes

A single node in our expression is represented by a trait `Exp` (Expression).

```
sealed trait Exp {  
  def value(bindings: Map[Variable, Double]): Double  
}
```

The only method in the trait is `value`. The purpose of this method is to calculate the value of an expression when the expression variables are given the desired values. As a parameter, the `Map` is a collection of variable-value pairs that the expression conveys further in the tree so that every single node in the tree will know them.

For the trait `Exp` we have decided to implement only four different nodes:

Variable

Describes a variable that has a name in a mathematical expression.

```
case class Variable(name: String) extends Exp {  
  def value(bindings: Map[Variable, Double]) = bindings.getOrElse(this, throw new UnknownValueException(this))  
}
```

Const

Describes a constant value, a number.

```
case class Const(c: Double) extends Exp {  
  def value(bindings: Map[Variable, Double]) = c  
}
```

Add

Describes the addition of two expressions (Exp). Note how the method value calculates its value from the value methods of the expressions it includes.

```
case class Add(u: Exp, v: Exp) extends Exp {  
  def value(bindings: Map[Variable, Double]) = u.value(bindings) + v.value(bindings)  
}
```

Mul

Describes the multiplication of two expressions (Exp).

```
case class Mul(u: Exp, v: Exp) extends Exp {  
  def value(bindings: Map[Variable, Double]) = u.value(bindings) * v.value(bindings)  
}
```

UnknownValueException

In addition, the class has an exception class where the expression can report problems in calculating its value.

```
case class UnknownValueException(v: Variable) extends Exception(v.toString + " could not be bound.")
```

Expressions

The described four nodes can be used to represent any math expressions with only multiplication and addition.

For example, a mathematical expression $1 + x + 3xy$ would be written with our nodes like this:

```
val x = Variable("x")  
val y = Variable("y")  
val e = Add(Const(1), Add(x, Mul(Const(3), Mul(x,y))))
```

In practice, the form is $(1 + (x + (3 * (x * y))))$

The value of the expression at point $x = 5, y = 3$ would be calculated and printed, as follows

```
println(e.value( Map( x->5, y->3 )))
```

Hint!!!!

In exercises 2-4 of this chapter it is important to effectively use the properties of the match structure shown in chapter 15.1. Match structure can be used to identify complex structures:

```
case Add(x, Add(y, z)) => // we do something with values of x,y,z
```

We can use structure called *pattern guard* to help use. It is an if statement that can be used to set an additional condition based on variables. Let us remind the Student class from section 15.1 and revise its method showPersonMore:

```
abstract class Person

case class Student(name: String, studentNumber: Int) extends Person

def isNew(studentId: Int) = studentId > 99999

def showPersonMore(person: Person): String = {
  person match {
    case Student(name, number) if isNew(number) => "freshman"
    case Student(name, number) => "Always freshman"
  }
}
```

When using pattern guard, there is no need to write a separate logic for identifying expressions within a case, for example, to simplify multiplication with zero (no simplification is done in exercises in this chapter). However, note the practicality of guard, for example, when constructing prettyprint.

```
case Mul(Const(a), b) if a == 0 => Const(0)
```

Remember to write statements so that you look at them from the most specific to the more general. For example, Add (e: Exp, f: Exp) should only be investigated after Add (v: Variable, e: Exp).

Exercise 3, Expressions 20p

You can find the object ExpressionTest in the given Expressions.scala file. Its code contains four simple mathematical expressions that you should write to the values of given variables in the given nodes. You can find them useful expressions as test cases in exercises 2, 3 and 4. Expressions and variables to which they are assigned and more detailed instructions can be found on the given program code.

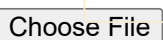
 This course has been archived (Thursday, 1 June 2023, 12:00).

You cannot submit this assignment

Exercise 3 (Expressions 1)

Select your files for grading

 Expressions.scala 

 No file chosen

Submit

Exercise 4, PrettyPrint 20p

In this exercise you should complement the given prettyprint method. Add lines to the method code to identify the cases described below, where **any** variable is multiplied to the second or third power, making the result more comfortable to read.

```
Mul(v, v) - - - > "v^2"
Mul(v, Mul(v, v)) - - - > "v^3"
Mul(Mul(v, v), v) - - - > "v^3"
```

Although there would be a lot of improve in the method (higher powers, merging additions within the same parenthesis, other ways to count second/third power, etc.), just make the requested changes so that we can evaluate the outcome.

The test run below shows what prettyprint prints.

```
val x = Variable("x")

val y = Variable("y")

println(prettyprint( Add( Mul(x, x), y) ))
( x^2 + y )

println(prettyprint( Mul(x, Mul(x, x)) ))
x^3

println(prettyprint( Mul(Mul(x, x), x) ))
x^3
```

 This course has been archived (Thursday, 1 June 2023, 12:00).

You cannot submit this assignment

Exercise 4 (PrettyPrint)

This course has been archived (Thursday, 1 June 2023, 12:00).
Select your files for grading



Expressions.Scala

Show anyway

Choose File No file chosen

Submit

Exercise 5, Partial evaluation (without value-method) 90p

In this task you should write a partial evaluation of the expression by assigning a value for one variable. In practice, your code should traverse the tree and replace the desired variable `v` with `Const` nodes including the given value `value`. Other variables remain unchanged.

Implement the method found on the given code `bind`.

```
def bind(exp: Exp, v: Variable, value: Double): Exp
```

The test run below shows how `bind` changes the expression.

```
val x = Variable("x")

val y = Variable("y")

val before = Add(x, Mul(Const(7), Add(y, x)))

println(before)
Add(Variable(x),Mul(Const(7.0),Add(Variable(y),Variable(x))))

println(bind(before, x, 3))
Add(Const(3.0),Mul(Const(7.0),Add(Variable(y),Const(3.0))))

println(bind(before, y, 2))
Add(Variable(x),Mul(Const(7.0),Add(Const(2.0),Variable(x))))
```

Tests up to 50p test whether the produced tree is correct. The last 10p is given if you manage to keep all the nodes as original ones whose children have not changed. In this case, the children should be processed first and then check whether the objects are still the same. Scala's operator `eq` compares whether two objects are the same object - mere similarity is not enough. It will definitely be useful for doing the exercise.

Loading assignment...

@ symbol in the match-structure

If you look at the code given for task 17.3, you will find the `@` character in the match structure.

```
case add @ Add(e: Exp, f: Exp) =>
  // Do something with variables add, e and f
```

Previously in section 15.1, we have become familiar with the syntax how the extractor assigned values to variables **e** and **f** which could then be used in the right side of the expression. Here, the **@** tag performs the same for the whole **Add** node. It assigns the variable **add** a reference to the whole node.

Exercise 6, Derivation 70p

In this exercise it is intended to perform symbolic derivation for the expression. It is enough to follow the instructions below.

```
D(constant , x ) -> 0
D( x      , x ) -> 1
D( y      , x ) -> 0
D( f + g , x ) -> D(f, x) + D(g, x)
D( f * g , x ) -> f * D(g, x) + g * D(f, x)
```

Write your implementation in the same file as in the previous tasks, by completing the method `derivate`. The method takes as a parameter a derivative and a variable to be derivatized.

```
def derivate(exp: Exp, v: Variable)
```

For example, the derivation of the summing node creates a new summing node and its children will be derivated versions of the original children. Since virtually all nodes change, you do not have to worry about the unchanged ones mentioned in the previous task for those nodes that do not change.

Variable argument list

There is an `apply` method in the companion object of `Case` class that can create objects from the class without using a constructor. The `Nodes.scala` file also contains a companion object for the `Add` class, with another `apply` method added to the class. This differs from the one produced by the case-class in terms of number and type of parameters.

```
object Add {  
  def apply(e : Exp* ):Exp = e.toList match {  
    case Nil      => Const(0)  
    case f :: Nil => f  
    case f :: g   => new Add(f, apply(g:_*))  
  }  
}
```

The most interesting parameter is its type: `Exp*`. The star after the type of parameter indicates that it is a so-called variable argument list (also called `varargs`), that is, the number of parameters is not predetermined. `Add` objects can now be created with the following expressions.

```
Add()  
> Const(0.0)  
  
Add(Const(0),Const(1),Const(0))  
> Add(Const(0.0), Add(Const(1.0), Const(0.0)))
```

When a method is called, the parameters are placed in a small array, which the method then gets as a parameter. Before processing with the `match` structure, the parameter `e` is converted to a list that is easier to handle. There are three case statements in the `match` structure:

Nil

`Nil` implies empty list.

f :: Nil

Here we use the list extractor, which decompresses the first item in the list to the left-hand variable of the colon `f` and the right side is compared with the empty list. So this case only happens with a list with one item.

f :: g

This differs from the previous one by assigning the end of the list to the variable `g`, which in this case can no longer be empty (previous cases handled it). Because we have both the element `f` and the list in `g`, we know that we need an `Add` node. Its left-hand item is `f`, but the right-hand side we get when we call our `apply` method for the rest of the list.

Here again, a new syntax is needed, which allows the list structure to be replaced by the function `varargs` list, so that each item in the list becomes a different parameter for the function. Adding the parameter after the notation `:_*` will cause the parameter `g` to be interpreted precisely in this way.

Feedback

Please note that this section must be completed individually. Even if you worked on this chapter with a pair, each of you should submit the form separately